Optimized Implementation for Exact Schedulability Tests of Real-Time Multiprocessor Schedulers

Artem Burmyakov[1][0000−0001−6787−0914], Makar Shevchenko[1], and Enrico Bini[2][0000−0001−9205−584X]

[1] Innopolis University, Russia
[2] University of Turin, Italy

Abstract. The problem of performing an exact schedulability test is central to guaranteeing the temporal correctness of real-time multiprocessor sporadic task systems. Several approaches were proposed, including the brute-force exploration of all possible execution schedules through a state-transition graph. These exact tests, however, do not scale efficiently with the size of a task set. The computational hardness of the exact schedulability test problem is a stiff barrier to solving various other scheduling challenges, such as an accurate performance comparison of existing multiprocessor schedulers.
Starting from Bonifaci's state-of-the-art enumeration-based exact test, we derive several code-level optimizations. Thanks to them, we achieve a test which is ten times faster and several times less memory demanding. We emphasize that our optimizations are not customized to any specific scheduler, differently than other approaches, which exploit scheduler-specific properties for pruning a system execution state space. Our improved test implementation enables thus an exact performance comparison of some well-known schedulers, which is also reported in this paper, and is the first result of this type in the literature. A particular result is the demonstration of the Global Least Laxity First (GLLF) scheduling performance dominance over other widely-studied Global Earliest Deadline First (GEDF) and Global Fixed Priority (GFP) schedulers.

Keywords: Real-time, Schedulability, Multiprocessor, Exact Test, Code Optimization

1 Introduction

A schedulability test is the key instrument to determine whether a real-time application is guaranteed to always fulfill its temporal requirements. An *exact* schedulability test is a necessary and sufficient condition for meeting these temporal requirements. Also, it allows to determine an exact number of processors required by an application. Considering that necessary tests may underestimate and sufficient tests may overestimate the processing requirements of a real-time application, there is clearly a strong need for an efficient exact test.

The derivation of an efficient exact test for multiprocessor scheduling is, however, a hard problem. The fundamental reason of this hardness is the lack of a so-called *worst-case scenario* for a real-time system. This leads to the need for examining a huge number of cases to verify that none of them violates system temporal requirements [1,5]. This enumerative approach is significantly advanced by schedulers-specific pruning methods [7], which however do not break the exponential runtime trend of this approach.

In face of this hardness, the research community has responded in two orthogonal directions:

- the development of simpler, sufficient-only tests [12,3,13,18], and
- the improvement of the efficiency of exact tests [1,5,6].

Our contribution aims at offering improvements to the implementation of any exact schedulability test. Our optimizations are not tailored to a specific scheduling policy. This enables the investigation of the performance of well-known real-time multiprocessor schedulers. In fact, today, despite the proliferation of many non-optimal multiprocessor scheduling algorithms, little is known about their relative performance. Even such elementary questions as the determination of the optimal priority assignment for the simplest GFP scheduler is still without an answer. The experiments enabled by the proposed optimizations reveal the performance superiority, in an average case, of the GLLF over other schedulers considered in our evaluation. This is the first exact evaluation of this type available in the literature.

1.1 Related works

The lack of a known worst-case jobs release scenario for multiprocessor real-time systems of both periodic and sporadic tasks implies that all proposed exact schedulability tests explicitly or implicitly enumerate possible jobs release scenarios [1,5,19,6,14,16,17,10,8,11]. The common steps are:

1. To assume discrete time and integer task parameters,
2. To enumerate all possible job releases, and
3. To exploit some conditions to terminate enumeration.

Normally, these mentioned approaches cannot naturally handle preemption. In fact, the preemption breaks the job in a number of segments which may vary for every job. Following this direction, several exact tests for periodic tasks were proposed [8,11].

A prominent approach to enumerate possible execution scenarios by means of a state-transition graph was proposed by Bonifaci and Marchetti-Spaccamela [5]. Their approach applies to both periodic and sporadic tasks, any scheduler, and it is easily adopted to more restrictive scheduling models, such as with limited preemptions and migrations of tasks between processors. Although their test dominates others, it does suffer from a high computation time and memory requirement, which we aim at reducing by proposing several code-level optimizations to their implementation.

1.2 Contributions

We propose the following contributions:

- A set of code optimizations for Bonifaci's exact test implementation [5], which noticeably reduce its runtime and memory consumption as shown later in Fig. 8. For example, in the case of 6 real-time tasks, the test becomes ∼40 times faster and half as memory consuming. Also, our code is publicly available.[3]
- The study of the relative performance gain for well-known multiprocessor schedulers GFP, GEDF, GLLF, and less known Global Earliest Deadline Zero Laxity (GEDZL). We confirm a better scheduling performance of GLLF, with respect to GFP and GEDF [15]. It is the first exact comparison of multiprocessor schedulers, according to our knowledge.

We remark that our optimizations do not reduce the theoretical complexity of the algorithm.

1.3 Paper organization

The paper is organized as follows:

Section 2 formalizes the models of real-time systems and multiprocessor schedulers;
Section 3 reviews Bonifaci's state-of-the-art exact schedulability test and its C++ implementation;
Section 4 proposes three code optimizations to Bonifaci's test implementation;
Section 5 adopts an improved test implementation to periodic tasks;
Section 6 evaluates the efficiency of our optimizations, as well as the relative performance gain of GFP, GEDF, GLLF, and GEDZL schedulers.

2 System Model

A real-time system is modeled by a set of *sporadic tasks* $\mathcal{T} = \{\tau_1, \ldots, \tau_n\}$. Each task is characterized by $\tau_i = (C_i, D_i, P_i)$ and releases a potentially infinite sequence of *jobs*, such that: C_i is the *execution time* of a τ_i job; D_i is the *relative deadline* of a τ_i job; P_i is the *minimal interarrival time* between consecutive τ_i job releases. We note that, although our analysis is illustrated for sporadic tasks, the proposed contributions apply to periodic tasks as well (see Section 5). Finally, C_i, D_i, P_i are integers, as well as $D_i \leq P_i$ (*constrained* deadlines). We assume discrete time with $t \in \mathbb{N} = \{0, 1, 2, ...\}$.

For a given time instant t, parameters c_i, d_i, p_i specify the current state of a task τ_i:

c_i – the remaining execution time of a pending job, if any;

[3] At `https://github.com/SyrexMinus/optimized_global_sched_test`

d_i – the remaining time until the job deadline, and
p_i – the remaining time until the next possible job release by τ_i, with $p_i = 1$ at $t-1$ indicating that τ_i may release a job at t.

We remark that:

- $D_i \leq P_i$ implies at most one pending job per task at a time;
- $c_i = 0$ implies no pending job for τ_i at a given time.

The *absolute deadline* of a τ_i job possibly pending at time t is at:

$$\overbrace{t + d_i}^{\text{absolute deadline}} = \overbrace{t - (P_i - p_i)}^{\text{release instant}} + D_i,$$

meaning that d_i is a function of p_i and task parameters D_i and P_i:

$$d_i = D_i - (P_i - p_i). \tag{1}$$

The *laxity* of a pending τ_i job is the maximum tolerable idle time without missing a deadline [1]:

$$\ell_i = d_i - c_i = D_i - (P_i - p_i) - c_i, \tag{2}$$

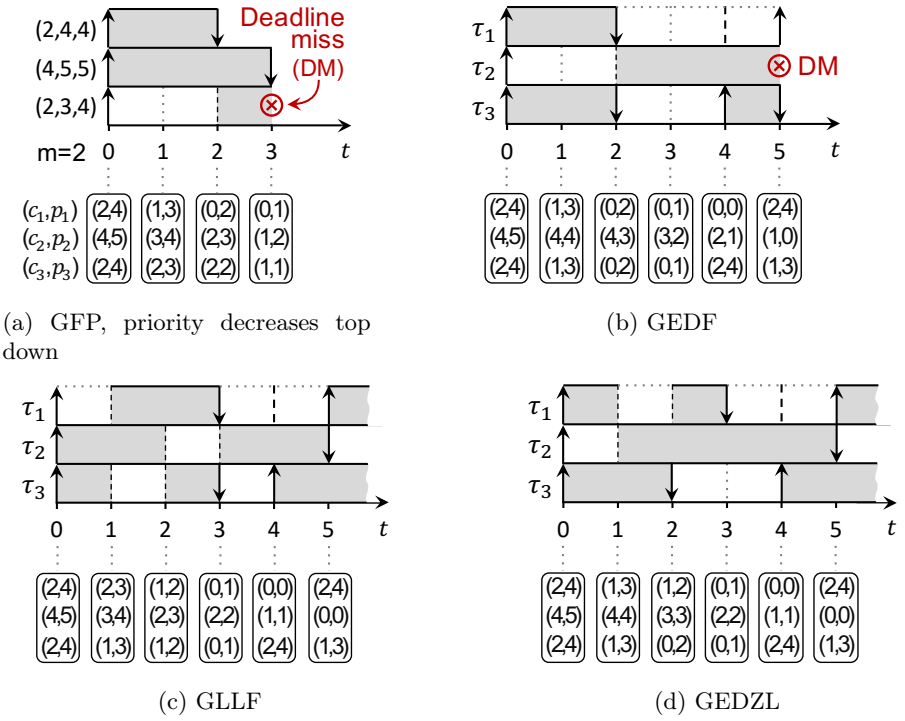
which can also be expressed through the only state variables p_i and c_i. Hence, from now on, we assume that only p_i and c_i, for all tasks τ_i, are part of the state.

Schedulers assign dynamically jobs to processors according to specific job priorities. In case of m identical processors, a scheduler selects at most m pending jobs. We assume a *work conserving* scheduler: no processor remains idle if any job has some pending work. We denote the *priority* of τ_i jobs by $\mathsf{pr}_i \in \mathbb{N}$. A larger value of pr_i denotes a higher priority for τ_i. Notice that the hypothesis $D_i \leq P_i$ implies the existence of at most one pending job per task. Hence, the assignment of priority pr_i to task τ_i means the priority of the only possibly pending job of τ_i. Priorities are *dynamic*, i.e. they may possibly change at every time unit. To define pr_i for all tasks, regardless of the presence of pending jobs, we assign

$$\mathsf{pr}_i = 0 \qquad \Leftrightarrow \qquad c_i = 0, \tag{3}$$

and we reserve positive priority values $\mathsf{pr}_i \geq 1$ for tasks with pending work ($c_i \geq 1$).

Our code optimizations are not restricted to a specific scheduler. Hence, below we review a few well-known multiprocessor schedulers — GFP, GEDF, GLLF, and GEDZL — which are later used for the evaluation of our optimizations in Section 6. In fact, we can represent any deterministic scheduler by the function $\mathcal{S}(x)$, which returns the subset of scheduled tasks for any given state $x = [(c_1, p_1), \ldots, (c_n, p_n)]$, with (c_i, p_i) defined earlier in this section. Figure 1 illustrates the definition of the four schedulers and the corresponding evolution of the states.



(a) GFP, priority decreases top down

(b) GEDF

(c) GLLF

(d) GEDZL

Fig. 1: Schedules for the same release scenario. Notation: upward arrow denotes a job release, downward arrow – completion, filled rectangle – CPU allocation, thick dashed line – earliest time for possible next release, rounded rectangles – states $[(c_1, p_1), (c_2, p_2), (c_3, p_3)]$. We remind that d_i is not represented in the state as it is a function of p_i through (1).

Global Fixed Priority (GFP) assumes jobs priorities to be fixed by a system designer and expressed through tasks ordering:

$$\mathsf{pr}_i > \mathsf{pr}_k \iff i < k \tag{4}$$

Global Earliest Deadline First (GEDF) dynamically determines jobs priorities based on their deadlines:

$$\mathsf{pr}_i > \mathsf{pr}_k \iff \begin{bmatrix} d_i < d_k \\ d_i = d_k \wedge i < k \end{bmatrix} \tag{5}$$

where "[" is a logical OR.

Global Least Laxity First (GLLF) instead is based on laxities:

$$\mathsf{pr}_i > \mathsf{pr}_k \iff \begin{bmatrix} \ell_i < \ell_k \\ \ell_i = \ell_k \wedge i < k \end{bmatrix} \tag{6}$$

Global Earliest Deadline Zero Laxity (GEDZL) gives the highest priorities to jobs with zero laxities, otherwise uses GEDF [2]:

$$\mathsf{pr}_i > \mathsf{pr}_k \iff \begin{bmatrix} \ell_i = 0 \wedge \ell_k > 0 \\ \ell_i = 0 \wedge \ell_k = 0 \wedge i < k \\ \ell_i > 0 \wedge \ell_k > 0 \wedge d_i < d_k \\ \ell_i > 0 \wedge \ell_k > 0 \wedge d_i = d_k \wedge i < k \end{bmatrix} \tag{7}$$

For any priority assignment pr_i, the set $\mathcal{S}(x)$ of scheduled tasks when at state x is defined by

$$\mathcal{S}(x) = \big\{\text{first } m \text{ higher priority tasks according to } \mathsf{pr}_i(x), \text{ with } \mathsf{pr}_i(x) \geq 1\big\}. \tag{8}$$

We recall that from Eq. (3), $\mathsf{pr}_i \geq 1$ if and only if the task τ_i has some pending work ($c_i \geq 1$).

In case of priority ties, we assign a higher priority to task τ_i with smaller i. This ensures a total priority order among pending jobs and then a unique definition of $\mathcal{S}(x)$ for any state x.

Later in Section 6 we report the relative scheduling performance of the four schedulers above. We observe no dominance of one over other schedulers: there are task sets schedulable by any of them, but unschedulable by all the others.

3 Background on an Exact Schedulability Test

Our code optimizations are built on top of the original Bonifaci's test [5]. Such a test is implemented in C++ and available on-line publicly[4]. The advantages of this test are

- it is applicable to any multiprocessor scheduler, and
- it significantly outperforms other available tests in terms of computation time and memory demand [6].

3.1 Bonifaci's exact test: a two players game

To test schedulability, Bonifaci's tool constructs a state-transition graph, with a set of *states* and *transitions* between them representing all possible execution scenarios. Fig. 2 illustrates examples of such graphs[5]. The tool implements the Breadth-First Search (BFS) to search for a failure state with a missed deadline, if any, in the graph. Alg. 1 outlines the key steps of Bonifaci's test, which we describe next.

At a high level, Alg. 1 is modeled as a two-players game. The two players are:

[4] `https://github.com/vbonifaci/sporadic`

[5] Tiny task sets enable us to depict entire graphs. Also, as GFP and GEDF graphs are nearly identical for such tiny task sets, we choose a different task set for each case.

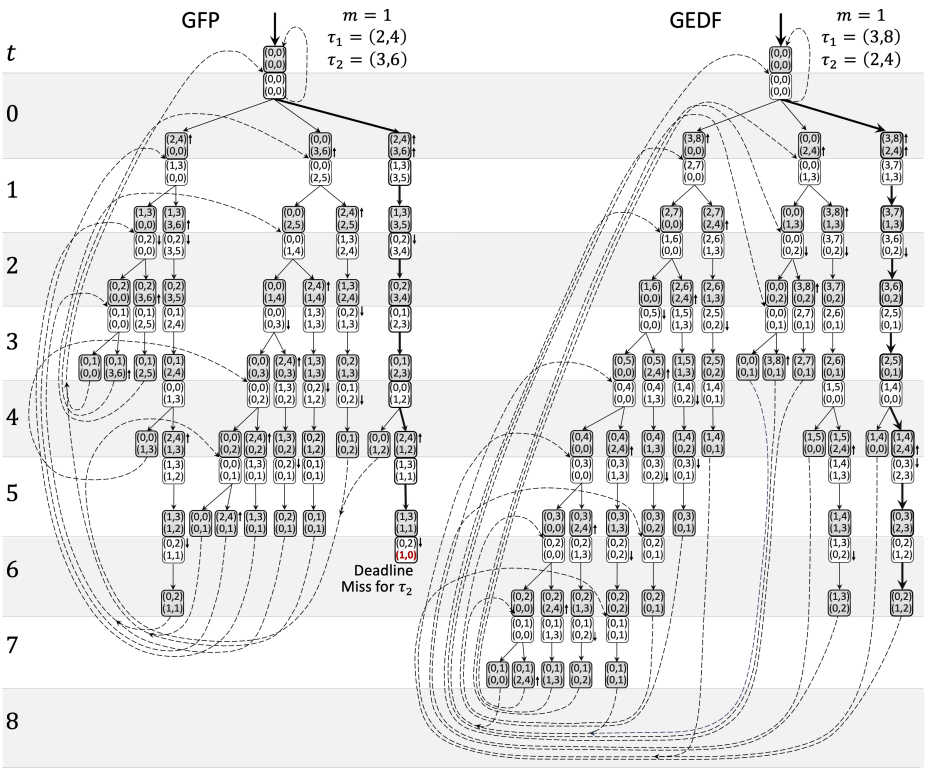


Fig. 2: Examples of state-transition graphs for GEDF and GFP schedulers. States "Rel" and "Sch" are represented with white and grey background, respectively. Solid arrows between states represent non-deterministic Sch → Rel transitions. Dashed-arrows transitions Rel → Sch represent Sch successors coincidence of Rel states explored at different algorithm iterations. Thick-arrow transitions correspond to periodic job releases. Job releases are represented by upward arrows beside a Sch state. Job completions are represented by downward arrows beside a Rel state. Horizontal thin lines represent clock ticks. Notice that the GFP task set is not schedulable, whereas the GEDF one is.

1. The player "scheduler" (Sch), which deterministically selects at most m tasks for execution according to a chosen scheduler, such as the ones defined in Equations (4)–(7). Also, it advances the time by one unit by decrementing state parameters in accordance to the scheduled tasks.
2. The player "releases" (Rel), which **may** release new task instances in accordance to the minimum interarrival time P_i.

Transitions between graph states are then alternating moves of two players.

State x is represented by the tuple

$$x = \left[(c_1, p_1), ..., (c_n, p_n), \mathsf{player}\right] \tag{9}$$

with c_i and p_i defined in Section 2, representing the remaining execution time and time to the earliest next release for task τ_i, respectively. Notice that the time until the deadline d_i is not explicitly included into Eq. (9), as d_i is the function of p_i through Eq. (1). The state x also includes the field $\mathsf{player} \in \{\textsc{Rel}, \textsc{Sch}\}$ denoting the type of state. If the field $x.\mathsf{player}$ is equal to Rel (resp. Sch), then the player Rel will make next transition with RelMove (resp. the player Sch will make a transition with SchMove). Each players makes a move from a state x to the next one x^+, as described below.

Scheduler move If $x.\mathsf{player}$ is equal to Sch, the scheduler operates a transition from x to $x^+ = \textsc{SchMove}(x)$. Such transition is defined by

$$\textsc{SchMove}(x) = x^+ = [(c_i^+, p_i^+)_{i=1}^n, \textsc{Rel}], \text{ with } \begin{cases} c_i^+ = c_i - 1 & \tau_i \in \mathcal{S}(x) \\ c_i^+ = c_i & \tau_i \notin \mathcal{S}(x) \\ p_i^+ = \max(p_i - 1, 0) & \forall \tau_i \end{cases} \tag{10}$$

Above, the pending execution time c_i is decremented by one, only for the scheduled tasks — the ones belonging to $\mathcal{S}(x)$, defined in Eq. (8) as the set of scheduled tasks. The time to the earliest next release p_i, and consequently the time until the deadline d_i due to (1), is always decremented by one unit until it reaches zero, to encode the elapsing of time. When p_i reaches zero, if $c_i > 0$ a deadline miss is detected.

The assumption of discrete time granularity by Bonifaci's test is incorporated in the SchMove function above. Thus, adaptation of the test to continuous time requires changes in SchMove. Alternatively, we could model transitions in a state-transition graph by means of constrained programming without relying on discrete time. These investigations, however, are planned as a future work.

Releases move After a scheduler's transition has occurred, a release transition takes place (that is the move of the Rel player) from a state x with $x.\mathsf{player} = \textsc{Rel}$. We model such a transition from x to $x^+ = \textsc{RelMove}(x, \mathcal{R})$ by the function RelMove, which also depends on a subset of tasks $\mathcal{R} \subseteq \mathcal{T}$. The subset $\mathcal{R}$ contains the tasks which are allowed to release new instances, that is the minimum interarrival time has elapsed. Formally, $x^+ = \textsc{RelMove}(x, \mathcal{R})$ is defined by

$$\begin{aligned} \textsc{RelMove}(x, \mathcal{R}) = x^+ &= [(c_i^+, p_i^+)_{i=1}^n, \textsc{Sch}] \\ \text{with } (c_i^+, p_i^+) &= \begin{cases} (C_i, P_i) & \text{if } p_i = 0 \ \wedge \ \tau_i \in \mathcal{R} \\ (c_i, p_i) & \text{otherwise.} \end{cases} \end{aligned} \tag{11}$$

We observe that the release of a τ_i task at a new state x^+ occurs only if the minimum interarrival time has elapsed by the state x^+ **and** the task τ_i belongs to the set $\mathcal{R}$. This was introduced by Bonifaci to model the sporadic nature of releases, in which a task might not release a new job despite the elapse of P_i time from the previous release. In fact, Bonifaci's Alg. 1 checks all possible sets $\mathcal{R} \subseteq \mathcal{T}$

of tasks to be released. Observe that the only source of non-determinism in the state-transition graph comes from the sporadic nature of task releases, which is modeled by the possible choices of $\mathcal{R} \subseteq \mathcal{T}$. If instead, we are considering periodic task releases only, we invoke $x^+ = \textsc{RelMove}(x, \mathcal{T})$, that is with $\mathcal{R} = \mathcal{T}$.

In Fig. 2, the transitions from x to $x^+ = \textsc{RelMove}(x, \mathcal{R})$ for all possible subsets $\mathcal{R} \subseteq \mathcal{T}$ are represented by solid arrows, from a white state x to a gray one x^+. For example, from the initial state $\big[(0,0),(0,0),\textsc{Rel}\big]$ there are four possible release scenarios corresponding to the four possible subsets $\mathcal{R}$ equal to $\{\tau_1, \tau_2\}$, $\{\tau_1\}$, $\{\tau_2\}$, and $\emptyset$. Instead, for any x with $x.\mathsf{player} = \textsc{Sch}$ depicted in gray in the figure, the following state $x^+ = \textsc{SchMove}(x)$ is attached at the bottom of x, since the transition $\textsc{SchMove}(x)$ is deterministic.

The goal of Bonifaci's algorithm is to traverse a state transition graph until a failure state x with a missed deadline is detected, that is:

$$\exists x, i: \quad d_i = 0 \ \wedge c_i > 0, \tag{12}$$

or until all graph states are visited with no failure state detected, that is the algorithm confirms a task set schedulability.

3.2 Bonifaci's test: an algorithmic view

Algorithm 1 outlines the key steps of Bonifaci's test. Two main data structures are declared at Line 3:

- Q is a FIFO queue of states pending to be explored, and
- V is a set of states explored at previous iterations.

The algorithm loops until Q is empty (at Line 4), that is no more states to be explored are available.

To manipulate the Q and V structures, Alg. 1 uses the functions:

- $\textsc{EnqueueFifo}(Q, x)$ appends a state x to the end of the queue Q;
- $\textsc{Dequeue}(Q)$ takes the first state from Q to explore its successors;
- $\textsc{InsertSorted}(V, x)$ adds a state x to V and keeps V ordered. As we will see next, the preservation of the order in V suggested by Bonifaci, leads to a performance loss.

If Q is not empty, the first state x is fetched from Q (Line 5). Next steps depend on the x type:

Rel For all possible subsets $\mathcal{R} \subseteq \mathcal{T} = \{\tau_1, \ldots, \tau_n\}$ at Line 7, which represents the tasks allowed to release, a new state $\textsc{RelMove}(x, \mathcal{R})$ is generated according to Eq. (11). The exploration of all these subsets $\mathcal{R} \subseteq \mathcal{T}$ implements the sporadic nature of task releases. The check for legal releases is done inside of the $\textsc{RelMove}(x, \mathcal{R})$. For every x, there are 2^n cases to be examined, where n is the $\mathcal{T}$ size. All states generated at this stage are of the Sch type and stored into both ordered structures V and Q at Lines 10 and 11.

Algorithm 1 Bonifaci's Test

```
1: function ExactTest
2:     x_init ← [(0,0)^n_{i=1}, Sch]                          ▷ The initial state
3:     Q ← {x_init}, V ← {x_init}                             ▷ Storage initialization
4:     while Q ≠ ∅ do                                         ▷ Breadth-first search
5:         x ← Dequeue(Q)
6:         if x.player = Rel then                             ▷ Tasks releasing
7:             for all R ⊆ T do          ▷ Testing all possible subsets R of T
8:                 x ← RelMove(x, R)                          ▷ Equation (11)
9:                 if x ∉ V then                              ▷ State not explored yet
10:                    V ← InsertSorted(V, x)
11:                    Q ← EnqueueFifo(Q, x)
12:                end if
13:            end for
14:        else                                  ▷ x.player = Sch: tasks scheduling
15:            x ← SchMove(x)                                 ▷ Equation (10)
16:            if ∃i : d_i = 0 ∧ c_i > 0 then
17:                return UNSCHEDULABLE                       ▷ Deadline miss detected
18:            else if x ∉ V then                             ▷ State not explored yet
19:                V ← InsertSorted(V, x)
20:                Q ← EnqueueFifo(Q, x)
21:            end if
22:        end if
23:    end while                                              ▷ All states explored
24:    return SCHEDULABLE
25: end function
```

Sch If a state x is of type Sch, the tasks are scheduled according to any given scheduling policy by invoking SchMove(x) at Line 15, which also advances the time by one unit according to Eq. (10). After that, every task is checked for a deadline violation at Line 17 and, if this is the case, the algorithm terminates by reporting the unschedulability of $\mathcal{T}$. Otherwise, the state generated after this scheduler transition is stored into V and Q at Lines 19 and 20. Note that there is always a unique successor generated by SchMove(x) at Line 15 because we assume that the scheduler is deterministic, whichever is the implemented policy.

Condition $x \notin V$ invoked at Lines 9 and 18 checks if the generated state x was already explored by the algorithm at previous iterations. If this is the case, it can be safely discarded. We refer to this condition as "graph loops detection". Such a condition is necessary. Omitting such a check will let the exploration grow unboundedly.

At Line 4, we check if the queue Q is empty. When this happens all possible graph states are explored, and the test terminates.

Figure 3 illustrates the key steps of Alg. 1. At the beginning (Figure 3a), the queue Q of the states to be visited and the structure V of visited ones are initialized with the initial state x_{init}. Such a state represents the absence of pending jobs and the possibility for any task to immediately release a job. Figure 3b shows the transition from a state of type SCH to the next state of type REL. Such a transition is made by the scheduler in accordance to Equation (10): the pending execution time of the scheduled tasks is decremented by one, and the time is advanced by one. Figure 3c shows that when a task has $p_i = 0$, then it may release a new job. Finally, Figure 3d shows that when the successor of any state was already visited, then the state expansion is stopped. Such a condition is detected by checking if next state belongs or not to V.

4 Proposed Code-Level Optimizations

We now describe our code-level optimizations of Bonifaci's Algorithm 1. As preliminary step, to identify the portions of code which would benefit from an improved implementation, we profiled the runtime of Alg. 1. Figure 4 shows the percentage of runtime per type of operation of the algorithm. Such profiling was conducted for the case of the GFP scheduler, but the key observations are valid for other well-known schedulers. Detailed experimental settings and evaluation procedure are described later in Section 6.

From the experiment of Figure 4, we gain the following insights:

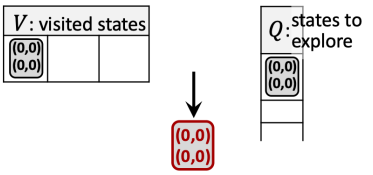
1. The $x \notin V$ condition takes most of the Alg. 1 runtime, with its runtime share increasing with the number of tasks.
2. The SCHMOVE function for simulating the scheduler decision takes time as it needs to fetch the state, to compute the scheduling decision, to modify the states accordingly, and then store them back.
3. The INSERTSORTED function is also time-consuming as preserving the order in a large set takes indeed time.

Based on the above observations, we propose the following code-level optimizations:

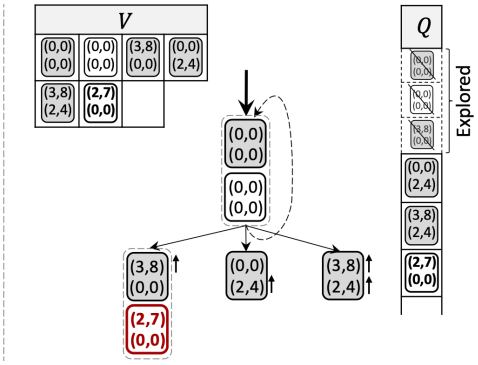
1. Optimization 1: the avoidance of the storage of irrelevant states into V, as the reduction of the memory consumed by the states is beneficial for all load, store, and search operations;
2. Optimization 2: the improvement of data structure for V: an unordered set with a custom hash function instead of an ordered map;
3. Optimization 3: a refined implementation for the SCHMOVE function.

4.1 Optimization 1: Reduced number of stored states

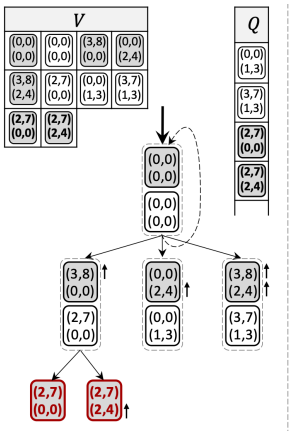
As observed earlier, the reduction of the number of stored states is beneficial to many aspects: from the memory usage to the time for searching. In fact, despite the exponential runtime, the algorithms's key practical limitation is the memory



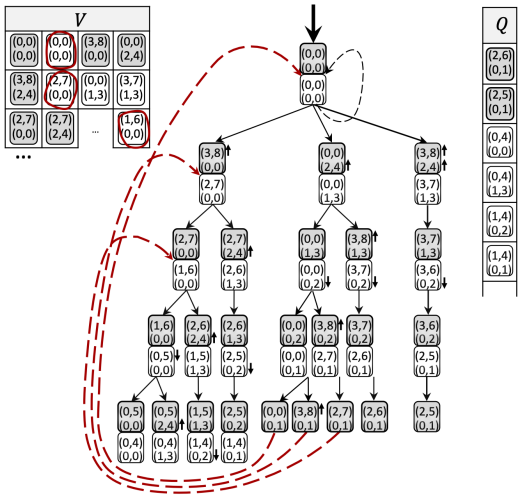
(a) Initialization of V and Q at Line 3 of Alg. 1.



(b) The SCH move at Line 15. In this case, task τ_2 is the only one with pending work ($c_2 \neq 0$) and it is then scheduled for execution. Afterwards, both its pending work c_2 and time to the next release p_2 are decreased by one.



(c) The REL move at Lines 7-13. Task τ_2 has $p_2 = 0$ then it may release a new job or not. In the transition to the left, it does not release any new job. In the transition to the right, it releases a new job, as indicated by the c_2 and p_2 values "refilled" to their initial values, as indicated by Eq. (11). This is also represented graphically by an upward arrow beside the task state.



(d) Detection of states coincidence at Lines 9 and 18

Fig. 3: The key steps of Alg. 1. Two tasks $\tau_1 = (3, 8, 8)$ and $\tau_2 = (2, 4, 4)$ are scheduled upon $m = 1$ processor by the GEDF scheduler. For simplicity of the representation, we show an implicit deadline case ($D_i = P_i$). The state of each task is represented by the pair (c_i, p_i) with c_i remaining execution time, and p_i remaining time to the earliest release. See Eq. (9) for more details on the state definition.

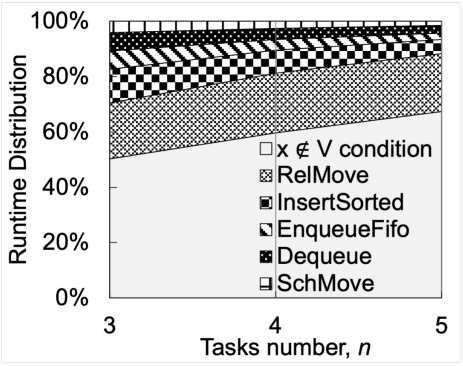


Fig. 4: Runtime profiling of the original Bonifaci's tool (Alg. 1).

requirement, which is proportional to the number of states. The system memory size provided by a typical computer is consumed within tens of minutes. After that, page swapping makes the performance drop to an unacceptable level.

The storage of states is needed for detecting graph loops, that is the coincidence of a generated state with some previously visited one, as shown visually in Figure 3d. This loop detection is essential for the algorithm termination: without it, the graph would grown unboundedly.

A first observation is that the scheduler is deterministic. Such a determinism is graphically represented by the unique transition from a Sch state to a Rel state (in Figure 2, from a gray state to a white state). We can then choose to use only one type of states and declare it as representative of both types. We select the Rel type of states, and from now on, we simply omit the type and use the following simplified representation

$$x = \big[(c_1, p_1), ..., (c_n, p_n)\big],$$

with c_i and p_i interpreted again as pending work of τ_i and remaining time to the earliest next release, respectively.

Algorithm 2 is the adaptation of Alg. 1, which uses one type of states only. The changes made are as follows:

- the stored states are now of one type only (the Rel type)
- the scheduler transition at Line 8 is made immediately, without storing the intermediate state.

Even making only these modifications yield a significant performance optimization:

- Nearly twice memory reduction for V;
- Runtime reduction for $x \notin V$ validation at Line 9, due to a smaller set V;
- Runtime reduction due to removing Lines 18 and 19 of Alg. 1.

A detailed runtime and memory gain is reported later in Section 6.

Algorithm 2 Optimized implementation of Alg. 1

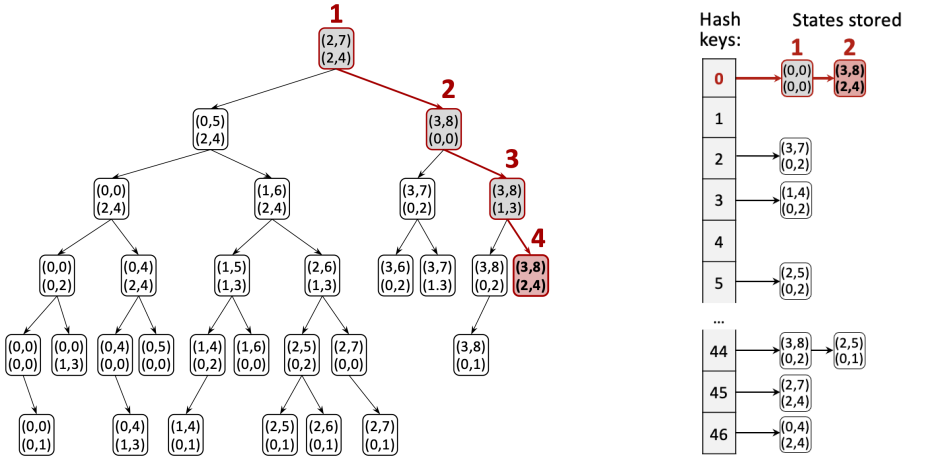
```
1: function OptimizedExactTest
2:     x_init ← [(0,0)]_{i=1}^n                ▷ Removed the type from the initial state
3:     Q ← {x_init}, V ← {x_init}                          ▷ Storage initialization
4:     while Q ≠ ∅ do                                       ▷ Breadth-first search
5:         x ← Dequeue(Q)
6:         for all R ⊆ T do                  ▷ Testing all possible subsets R of T
7:             x ← RelMove(x, R)
8:             x ← SchMove(x)                ▷ Make the Sch transition immediately
9:             if ∃i : d_i < c_i then
10:                return UNSCHEDULABLE                  ▷ Deadline miss detected
11:            else if x ∉ V then                        ▷ State not explored yet
12:                V ← InsertSorted(V, x)
13:                Q ← EnqueueFifo(Q, x)
14:            end if
15:        end for
16:    end while                                         ▷ All states explored
17:    return SCHEDULABLE
18: end function
```

4.2 Optimization 2: Hash table for states storage

Our second optimization reduces the computation time for the $x \notin V$ condition at Line 11 of Alg. 2. Recall that such a condition detects graph loops and is essential for the termination. The validation of this condition, however, is the most time consuming step of the algorithm, with its share of more than 60% of runtime, as reported in Fig. 4. One reason for such a high computational cost is a large size of V, which we cannot really mitigate. On the other hand, we can operate on the data structure used to store V. The original Bonifaci's implementation uses the STL C++ `map` structure for V. The `map` structure, however, has two significant shortcomings:

- It stores key-value pairs. This is however not necessary for us, as a set-like data structure is sufficient.
- It keeps elements sorted based on the red-black trees principle. In our case, however, the provided sorting does not yield the best performance.

To understand the issues with an ordered tree structure, consider Fig. 5a. It visualizes an example of V implementation by a red-black tree structure. In this tree checking the presence of the state $x = [(3, 8), (2, 4)]$ requires four comparisons labeled from "1" to "4". In general, search and insertion operations for tree structures have the $O(\log |V|)$ complexity. An alternative approach to a tree is a hash table, which in our case can be much more efficient, as the cardinality of V is in the order of millions. Hash tables support insertion and search operations at the $O(1)$ complexity, in an *average case*, although in a theoretical worst-case it can take up to $O(|V|)$.



(a) Red-black (balanced) tree to preserve the ordering of states (original implementation).

(b) Hash table to enable faster search and insertion (our implementation).

Fig. 5: Representation of the set V by using different data structures, for the task set $\tau_1 = (3, 8)$ and $\tau_2 = (2, 4)$ scheduled by GEDF.

Hash tables rely on the definition of a *hash function*, which we denote by $h_n(x)$. Such a function $h_n(x)$ takes the state x as input and returns the index of a *bucket*, at which the state x is to be stored. In the best case, the *bucket* $h_n(x)$ is empty before the insertion of x, while in other cases it can be already occupied by some other x' with $h_n(x) = h_n(x')$. This is called *collision*, in the terminology of hashing. The performance of a hash table depends significantly on the choice of a hash function: the more collisions it creates, the less efficient it is. Still, a hash function must not require too heavy computations, otherwise the benefits of hashing are lost in computing the function itself.

We have evaluated the performance of two hash functions. The first hash function $h_n^{\mathsf{full}}(x)$ is implemented by the following recurrent definition:

$$\begin{cases} h_1^{\mathsf{full}}(x) = x.c_1(P_1 + 1) + x.p_1 \\ h_i^{\mathsf{full}}(x) = \left(h_{i-1}^{\mathsf{full}}(x)(C_i + 1) + x.c_i\right)(P_i + 1) + x.p_i, \qquad i = 2, \ldots, n. \end{cases} \tag{13}$$

It has the property of assigning a different bucket to any possible state, from $h_n^{\mathsf{full}}(\mathbf{0}) = 0$ to the largest bucket id $h_n^{\mathsf{full}}([(C_i, P_i)]_{i=1}^n) = \prod_{i=1}^n (C_i+1)(P_i+1)-1$. A significant drawback of the $h_n^{\mathsf{full}}(x)$ function is its requirement to make $2n-1$ multiplications and $2n-1$ additions, causing significant computation overheads.

The second evaluated hash function $h_n^{\mathsf{bits}}(x)$ overcomes the drawback of $h_n^{\mathsf{full}}(x)$ by using only bitwise operations. It follows the same logic as h_n^{full}, except that it uses left bit shifts instead of multiplications and logical XOR instead of addition. Because of its code oriented nature, we find more convenient to represent it as a code fragment in Listing 1.1. In such implementation every multiplication by

Listing 1.1: C/C++ code fragment for the implementation of the $h_n^{\mathsf{bits}}(x)$ hash function

```
1  /* Assumptions in the code: */
2  /* (i) the "ts" variable contains task set parameters, */
3  /* (ii) the "x" state variable is an array of "struct" */
4  /*      instances, with each one representing states of */
5  /*      individual tasks */
6  /* (iii) "bitsC[i]" and "bitsP[i]" fields of "ts" are */
7  /*       bit sizes for binary representation of values */
8  /*       "ts.C[i]+1" and "ts.P[i]+1" */
9
10 unsigned int hash_bits(state * x) {
11     /* hash key initialization */
12     unsigned int id = x[0].c*(ts.P[0]+1) + x[0].p;
13
14     /* iteration over all tasks' states */
15     for (unsigned i = 1; i < ts.num; i++) {
16         id <<= ts.bitsC[i];    // binary shift-left
17         id ^= x[i].c;          // quick binary addition
18         id <<= ts.bitsP[i];
19         id ^= x[i].p;
20     }
21     return id;
22 }
```

any b of Eq. (13) is replaced by a multiplication by $2^{\lceil \log_2 b \rceil}$, which can then be efficiently implemented by left-shifting the representation by $\lceil \log_2 b \rceil$ bits. Then, the sum of addenda with non-overlapping representations, is efficiently implemented by the logic XOR (eXclusive OR). The down side of such hash function $h_n^{\mathsf{bits}}(x)$ is that the space of the represented states is expanded to the size

$$\prod_{i=1}^{n} 2^{\lceil \log_2 (C_i+1) \rceil} 2^{\lceil \log_2 (P_i+1) \rceil}$$

which is larger than the size $\prod_{i=1}^{n}(C_i+1)(P_i+1)$ of the space of the hash function $h_n^{\mathsf{full}}(x)$. Later, in Section 6.1 we report our evaluation of both functions.

This optimization then results in the replacement of Lines 11 and 12 of Alg 2 with accesses to V implemented via a hash table.

4.3 Optimization 3: A faster states computation

The last proposed optimization addresses the computation of all possible next states, in accordance to the possible job releases (due to the sporadic nature of tasks). According to the runtime profiling of Fig. 1, states computation takes at least 20% of its runtime. This computation can be however significantly optimized, as discussed below.

In Bonifaci's implementation, at Line 6 of Algorithm 2 all possible subsets $\mathcal{R}$ of $\mathcal{T}$ are generated. This large number of subsets generates a large number of invocations of the function RelMove.

This procedure performs poorly, as it requires to invoke RelMove() $2^{|\mathcal{T}|}$ times for every state x. For example, state $[(1,3),(3,5)]$ has the only one valid successor, but RelMove() is invoked four times, to confirm three remaining candidates to be invalid. To optimize this step of the generation of next state, we restrict the set to the only tasks which may release a job. Formally, we define

$$\mathcal{T}(x) = \{\tau_i \in \mathcal{T} : x.p_i = 0\} \tag{14}$$

and then we replace $\mathcal{R} \subseteq \mathcal{T}$ with $\mathcal{R} \subseteq \mathcal{T}(x)$ at Line 6 in Algorithm 2. Omitting C++ technical details, the definition above is implemented by using bitwise "AND" and shift operators.

5 Test Adaptation to Periodic Tasks

Alg. 2 is designed to test schedulability of sporadic tasks. This test however can be easily adapted to periodic tasks as well, with releases occurring as early as possible. The sporadic nature of tasks is incorporated into the RelMove function, which implements (11), and thus, to adapt it to periodic tasks, we apply the following changes:

1. Line 7 is removed;
2. In Line 8, the argument $\mathcal{R}$ is removed from the RelMove function inputs;
3. Eq. (11) is replaced with

$$\textsc{RelMove}(x) = x^+ = [(c_i^+, p_i^+)]_{i=1}^n, \text{ with } (c_i^+, p_i^+) = \begin{cases} (C_i, P_i) & \text{if } p_i = 0 \\ (c_i, p_i) & \text{otherwise.} \end{cases} \tag{15}$$

These changes are due to the fact that, unlike sporadic tasks, there is always a single neighbor successor for any graph state in case of periodic tasks. We emphasize that all proposed code-level optimizations remain valid for periodic tasks.

6 Evaluation

Finally, we report the evaluation results for our code-level optimizations to Alg. 1. In general, we achieve a runtime gain of more than 20–30 times, and we reduce the memory consumption by a factor of about two.

Hardware settings: We run experiments on a hardware platform with specification:

- Processor: 2.6GHz 6-core Intel Core i7
- System memory: 16GB (2x 8GB 2667MHz DDR4)
- Operating system: MacOS 14.4.1 (23E224)
- Compilation string: `g++ -ansi -std=c++17 -O3`

Task sets generation: The synthetic task sets given in input are randomly generated by using a publicly available[6] C++ task set generator. Such a method employs the CPLEX software[7] for solving integer-linear optimization problems, necessary to find the parameters of task sets, which fulfill any given configuration settings. Technical details regarding this generation procedure are available in [7]. Other existing task generators, such as [4] and [9], are not applicable to our case, due to a limited scale of integer task parameters.

Table 1 outlines the default parameter values. In every experiment, the value of one parameter is varied and shown along the x-axis of the corresponding plot. The other parameters remain fixed to default values. When generating the task sets, we allow a difference of up to 1.5% between the nominal task set density $U_{\mathcal{T}}$ of Table 1 and the generated value. This is necessary to speed-up the generation process by CPLEX.

Table 1: Default values of key parameters

Parameter	Value	Value (Fig. 10a)	Value (Fig. 9)
Processors number, m	2	2	3
Default scheduler	GEDF	GEDF	GEDF
Number of tasks, n	6	6	7
Task set density, $U_{\mathcal{T}} = \sum_{i=1}^{n} {}^{C_i}/_{P_i}$	1.6	2	2.5
Min. task deadline, $D_{\min}$	5	4	5
Max. to min. task deadlines, ${}^{D_{\max}}/_{D_{\min}}$	4	4	4
Periods to deadlines ratio, ${}^{P_i}/_{D_i}$	1	1	1
Min. number of task sets per data point	30	30	30

6.1 Hash functions

The choice of the hash function is a key element in the efficiency of the algorithm. In this section, we evaluate several possible design choices related to hash functions.

In the first experiment, shown in Figure 6, we investigate the appropriate number of buckets to store the set of visited states. By default, the C++ STL starts with a small number of buckets, increasing this number at run time when the average number of items per bucket exceeds a given threshold, in a process called *rehashing*. This operation is expensive, as it requires moving all elements in the set to new buckets. To reduce this runtime cost, we investigate pre-allocating the maximum number of buckets before starting the algorithm.

Figure 6 presents the results of this evaluation. We use the hash function $h_n^{\mathsf{full}}(x)$ from Eq. (13). For this function, the state space has size $\prod_{i=1}^{n}(C_i +$

[6] `https://github.com/burmyakov/Task-set-generator`
[7] `https://www.ibm.com/products/ilog-cplex-optimization-studio`

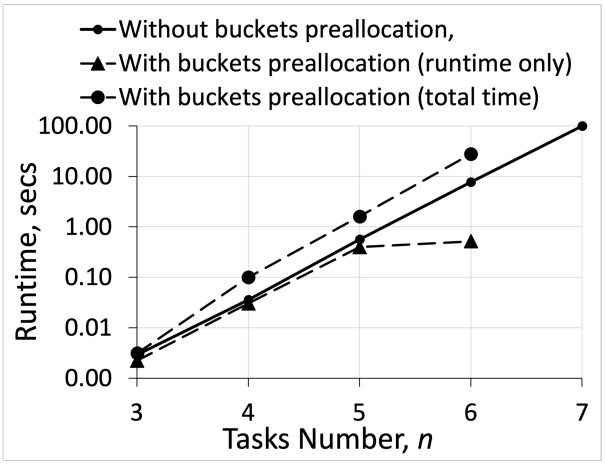
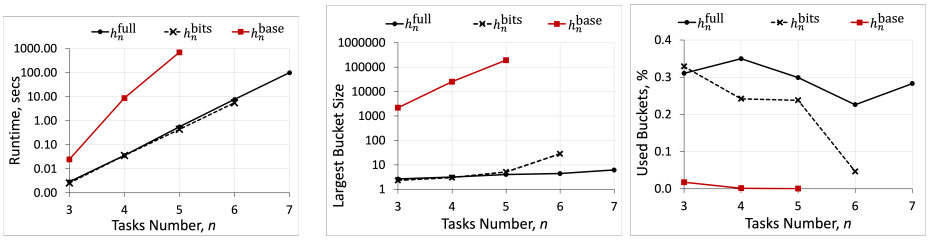


Fig. 6: Evaluation of the pre-allocation of buckets. For the case of pre-allocated buckets before start of the scheduling algorithm, we report the total time (circles, dashed) and the runtime excluding the pre-allocation phase (triangles, dashed). For comparison, we also overlap the case with no pre-allocation (dots, solid).

$1)(P_i + 1)$. The experiment indicates that if the number of buckets is large enough, the runtime of the algorithm (plot by triangles and dashed line in the figure) is indeed lower than letting the library do the rehashing (small dots, solid line). However, if we account also for the preliminary preparation of a proper number of buckets, such a gain is lost as the cost of pre-allocation is significant and growing with n. Hence, from now on, we simply delegate bucket creation and the rehashing to the STL implementation of the `unordered_set` C++ class.

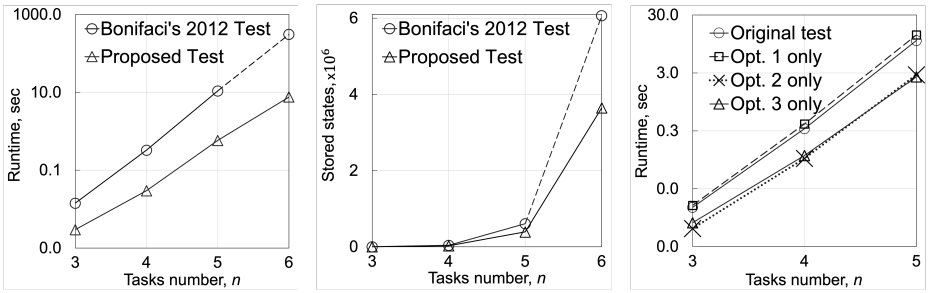
In the second experiment, we tested the two hash functions $h_n^{\mathsf{full}}(x)$ and $h_n^{\mathsf{bits}}(x)$ described in Section 4.2. To show the importance of hashing, we compare them over a constant hash function $h_n^{\mathsf{base}}(x) = 0$, which always generates collisions and then has linear complexity in the number of elements in the set.

For all three hash functions $h_n^{\mathsf{base}}(x)$, $h_n^{\mathsf{full}}(x)$, and $h_n^{\mathsf{bits}}(x)$, the experiments of Figure 7 show the runtime, the largest size of buckets, and the fraction of used buckets, for varying number of tasks. Figure 7a shows that, not surprisingly, hashing improves the runtime by increasing orders of magnitudes. A closer look confirms that $h_n^{\mathsf{bits}}(x)$ is more efficient than $h_n^{\mathsf{full}}(x)$. When the number of tasks n grows to 6 and beyond, the hash function $h_n^{\mathsf{bits}}(x)$ starts to generate more collisions, as indicated by the marked growth of the "Largest Bucket Size" of $h_n^{\mathsf{bits}}(x)$ in Fig. 7b. In fact, with this many tasks, the 32 bits used to represent the `id` of the state x of Listing 1.1 starts to be saturated and the shift operations simply shift bit out of the representation. In short, we start to pay for the greater computational efficiency of $h_n^{\mathsf{bits}}(x)$. The experiment of Figure 7c shows the percentage of used buckets compared to the available ones. It is striking to observe that only about 3 out of a thousand buckets are used. Also, the plot of



(a) Runtime performance (b) Size of largest bucket (c) Used buckets fraction

Fig. 7: Evaluating the hash functions, for 2 processors



(a) Total runtime gain (b) Total memory gain (c) Impact of each optimization

Fig. 8: Runtime and memory gain evaluation, as function of the number of tasks, for 2 processors

h_n^{base} shows that as only one bucket is used, the fraction of used buckets inevitably tends to zero as n and then the total number of buckets grows.

The conclusion of this evaluation is that $h_n^{\text{bits}}(x)$ may be slightly preferred with very few tasks, whereas $h_n^{\text{full}}(x)$ is preferable in more general settings. Hence, in the remainder of the experiments, we are always using $h_n^{\text{full}}(x)$.

6.2 Runtime and memory gain of the proposed optimizations

In this section, we illustrate the experiments evaluating speed-up provided by the optimized schedulability test, compared to the original Alg. 1. When making this evaluation, the scheduler was always GEDF, as the choice of the scheduler did not influence the efficiency gains.

As a number of tasks n is the major parameter affecting the test runtime, in all experiments illustrated in this section we varied n. Fig. 8a shows the gain of our proposed test over Bonifaci's original implementation, such a gain grows with the number of tasks, reaching a factor around 50 when $n = 6$. Fig. 8b illustrates that the number of stored states is significantly less, bringing a memory consumption reduced by half. We also evaluated the contribution of each

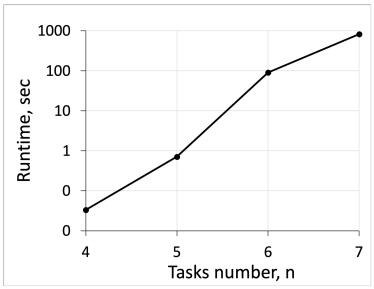
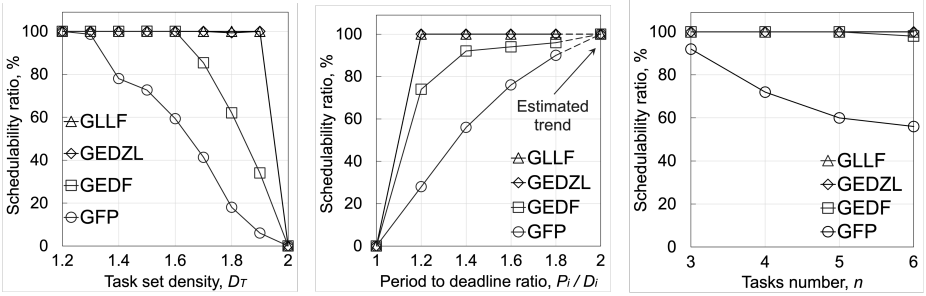


Fig. 9: The test runtime for $m = 3$ processors



(a) Varying task set density. (b) Varying ratio of task periods to deadlines. (c) Varying tasks number.

Fig. 10: Scheduling efficiency for GLLF, GEDZL, GEDF, and GFP schedulers, for 2 processors

individual optimization into a total runtime reduction, with the results reported in Fig. 8c. We notice that, unlike Optimizations 2 and 3, Optimization 1 does not yield any runtime gain, but instead a twice memory reduction reported in Fig. 8b.

To check the test scalability, we also examined it for $m = 3$ processors and a varying number of tasks n, with all optimizations being enabled. The results are reported in Fig. 9.

6.3 Comparison of schedulers performance

We conclude by evaluating the exact performance of a few schedulers. We are not aiming at comparing the vast literature on sufficient schedulability tests. More simply, supported by our efficient implementation of Bonifaci's test, we are addressing the question: what is the scheduler which can schedule most of the task set among the non-optimal ones?

To adapt the exact test to any scheduler, the only needed effort is a proper definition of the function $\mathcal{S}(x)$ returning the set of scheduled tasks, when the system is in state x. Hence, the availability of our more efficient implementation

enables a comparison of the scheduling efficiency of several schedulers described in Section 2, for small task systems.

We compare GFP, GLLF, GEDF, and GEDZL. For the GFP scheduler, we use deadline-monotonic priorities, with the highest priority assigned to a task with the smallest deadline D_i. Evaluation results are reported in Fig. 10, for a varying total task set density $U_{\mathcal{T}}$, ratio of tasks periods to deadlines[8] P_i/D_i, and the number of tasks n. For the experiment in Fig. 10b, the test has not succeeded to terminate for some task sets due to limited system memory. For the selected evaluation settings of Table 1, we observe an interesting trend, that in an average case GFP and GEDF schedulers are outperformed by GLLF and GEDZL. The superior performance of GLLF are explained by its greater distribution of remaining workload among the tasks. Instead, the observation of the top performance of GEDZL is, to the best of our knowledge, original. Also, our implementation enables a comparison of the accuracy of the many existing sufficient schedulability tests, which is not the focus of our paper, however.

We finally remark that we did not observe a significant impact of other task set parameters on the schedulability performance.

7 Conclusion

We provided three code-level optimizations to a publicly available exact schedulability test by Bonifaci and Marchetti-Spaccamela, which applies to multiprocessor real-time sporadic task systems. The derived optimizations resulted in a significantly faster test with a runtime gain exceeding 20–30 times, as well as twice memory reduction of consumed memory. Considering that the test applies to various real-time schedulers, we also evaluated the scheduling efficiency of GLLF, GEDF, GEDZL, and GFP schedulers. For chosen experimental settings, the result demonstrates a better performance of GLLF and GEDZL, in an average case.

Acknowledgment The authors are very grateful to the reviewers for the high quality comments and the suggested improvements.

References

1. Baker, T.P., Cirinei, M.: Brute-force determination of multiprocessor schedulability for sets of sporadic hard-deadline tasks. In: International Conference On Principles Of DIstributed Systems, pp. 62–75 (2007)
2. Baker, T.P., Cirinei, M., Bertogna, M.: EDZL scheduling analysis. Real-Time Systems **40**, 264–289 (2008)
3. Bertogna, M., Cirinei, M., Lipari, G.: Schedulability analysis of global scheduling algorithms on multiprocessor platforms. IEEE Transactions on parallel and distributed systems **20**(4), 553–566 (2008)

[8] For some task sets in Fig. 10b the test could not terminate due to limited memory. This is indicated by the "estimated trend" label.

4. Bini, E., Buttazzo, G.: Measuring the performance of schedulability tests. Real-Time Systems Journal **30** (2005)
5. Bonifaci, V., Marchetti-Spaccamela, A.: Feasibility analysis of sporadic real-time multiprocessor task systems. Algorithmica **63**, 763–780 (2012)
6. Burmyakov, A., Bini, E., Lee, C.G.: Towards a tractable exact test for global multiprocessor fixed priority scheduling. IEEE Transactions on Computers **71**(11), 2955–2967 (2022)
7. Burmyakov, A., Bini, E., Tovar, E.: An exact schedulability test for global FP using state space pruning. In: Proceedings of the 23rd International Conference on Real Time and Networks Systems, pp. 225–234 (2015)
8. Cucu-Grosjean, L., Goossens, J.: Exact schedulability tests for real-time scheduling of periodic tasks on unrelated multiprocessor platforms. Journal of systems architecture **57**(5), 561–569 (2011)
9. Davis, R., Burns, A.: Priority assignment for global fixed priority pre-emptive scheduling in multiprocessor real-time systems. In: Proceedings of the 30th IEEE Real-Time Systems Symposium (2009)
10. Foughali, M., Mikučionis, M., Zhang, M.: Scalable computation of inter-core bounds through exact abstractions. In: International Conference on Computers, Software, and Applications (COMPSAC) (2024)
11. Gohari, P., Voeten, J., Nasri, M.: Reachability-based response-time analysis of preemptive tasks under global scheduling. In: 36th Euromicro Conference on Real-Time Systems (ECRTS 2024). Schloss Dagstuhl–Leibniz-Zentrum für Informatik (2024)
12. Goossens, J., Funk, S., Baruah, S.: Priority-driven scheduling of periodic task systems on multiprocessors. Real-time systems **25**, 187–205 (2003)
13. Guan, N., Stigge, M., Yi, W., Yu, G.: New response time bounds for fixed priority multiprocessor scheduling. In: 2009 30th IEEE Real-Time Systems Symposium. pp. 387–397. IEEE (2009)
14. Larsen, K.G., Larsson, F., Pettersson, P., Yi, W.: Efficient verification of real-time systems: Compact data structure and state-space reduction. In: Proceedings Real-Time Systems Symposium. pp. 14–24. IEEE (1997)
15. Lee, J., Easwaran, A., Shin, I.: Laxity dynamics and LLF schedulability analysis on multiprocessor platforms. Real-Time Systems **48**, 716–749 (2012)
16. Nasri, M., Brandenburg, B.B.: An exact and sustainable analysis of non-preemptive scheduling. In: 2017 IEEE Real-Time Systems Symposium (RTSS). pp. 12–23. IEEE (2017)
17. Ranjha, S., Gohari, P., Nelissen, G., Nasri, M.: Partial-order reduction in reachability-based response-time analyses of limited-preemptive dag tasks. Real-Time Systems **59**(2), 201–255 (2023)
18. Sun, B., Kloda, T., Caccamo, M.: Response time analysis for fixed-priority preemptive uniform multiprocessor systems. In: 36th Euromicro Conference on Real-Time Systems (ECRTS 2024). Schloss Dagstuhl–Leibniz-Zentrum für Informatik (2024)
19. Sun, Y., Lipari, G.: A pre-order relation for exact schedulability test of sporadic tasks on multiprocessor global fixed-priority scheduling. Real-Time Systems **52**, 323–355 (2016)